

# Reiy Finance: Intent-based Meta DEX Aggregator

v2.0 - June, 2026

Wyner Hoang

Reiy Finance

bach.hd.28@gmail.com

## ABSTRACT

The DeFi ecosystem on Sui has matured into a full stack of decentralized exchange models, spanning automated market makers, a native central limit order book in DeepBook, and aggregators, all serving a large user base on a platform built for parallel execution, low latency, and cheap fees. Yet regardless of the model, liquidity remains fragmented across independent protocols, and slippage stays an inherent cost that every venue tries to minimize but cannot remove. At the same time, existing intent markets on other ecosystems impose high entry costs on the agents that fill orders, which leads to oligopoly and erodes the very execution quality that users should receive.

We present Reiy, the first intent based meta DEX aggregator on Sui. Rather than selecting a pool or a route, a user submits an intent whose assets are escrowed on chain, and a permissionless network of solvers competes in epoch gated auctions to fill it at the best price by matching a coincidence of wants and by sourcing external liquidity. Prevailing intent designs keep this coordination off chain to preserve low latency and cost; Reiy instead escrows and enforces intents on chain while retaining that profile, by confining the off chain coordinator to selection and signing.

Reiy adapts the design of CoW Protocol to Sui through three contributions. First, the SBBO admission floor reads an on chain reference price and rejects any intent whose minimum falls below the market floor. Second, permissionless solver competition removes the entry barriers that drive intent markets toward oligopoly. Third, settlement exploits Sui Programmable Transaction Blocks and the hot potato pattern to bind the off chain certificate to a single atomic on chain transaction, so escrowed assets move only when every protection holds. Reiy neither competes with nor replaces any venue on Sui; it is an execution layer that turns the ecosystem's fragmented liquidity into depth a single trade can reach.

## 1 INTRODUCTION

### 1.1 A Mature DeFi Stack on Sui

Sui hosts a mature DeFi stack, spanning automated market makers, a native central limit order book in DeepBook, aggregators, lending, and derivatives, on an object centric, parallel execution base with low latency and cheap fees [5].

The ecosystem does not lack places to trade; it lacks a layer that turns this breadth into one efficient market rather than a set of isolated venues.

### 1.2 Fragmentation and Slippage

Liquidity is fragmented: each venue prices a pair against its own partial view of depth, so capital that could clear a trade jointly sits divided, and a trader seeking best execution must compare venues and split orders by hand. Fragmentation also fixes a floor on slippage that no venue can remove while its liquidity stays siloed; the deeper the split, the worse the realized price and the more value leaks to arbitrage and reordering rather than to the user.

### 1.3 Why Existing Intent Markets Fall Short

Intent based trading addresses this: a user states a desired outcome, such as a minimum to receive, and delegates execution to competing agents. CoW Protocol demonstrates the design on Ethereum, matching opposing orders directly and aggregating external liquidity to beat single venue execution [3]. Its weakness is competition itself: solver participation costs restrict entry and can drive the market toward oligopoly, eroding the execution quality intents promise [2]. A viable intent system must therefore make permissionless solver participation a first class goal. None exists natively on Sui, and Ethereum designs do not transfer to its object centric model.

### 1.4 Reiy

Reiy, the first intent based meta DEX aggregator on Sui, adapts the design of CoW Protocol [3] to a chain whose fast finality and low fees let the auction and settlement loop run at a cadence Ethereum cannot match. It is deliberately not another DEX: rather than competing with the venues already on Sui, it sits above them as an execution layer that

turns their fragmented liquidity into depth a single trade can reach.

The remainder of this paper develops the design. Section 2 fixes the preliminaries and the research question, Section 3 the system model, and Section 4 the architecture. Section 5 defines the core constructs: intents, the SBBO floor, solvers, certificates, and the Programmable Transaction Block settlement that binds them. Section 6 describes the off chain coordinator, Section 7 the economic model and the REIY token, and Section 8 the resulting security properties.

## 2 BACKGROUND AND PRELIMINARIES

This section fixes the primitives and the research question; readers fluent in Sui and intent trading may skip ahead.

### 2.1 The Sui Execution Model

Reiy relies on two features of Sui’s object model [5]. A *shared object* holds escrowed user assets that any party may act on under contract enforced rules. And a Programmable Transaction Block (PTB) composes several Move calls into one atomic transaction, where Move’s linear *hot potato* type forces a value created mid block to be consumed before the block ends. Together they let Reiy escrow funds on chain and settle a batch in a single all-or-nothing transaction; Section 5.6 develops the hot potato in detail.

### 2.2 Liquidity and the On-Chain Price Reference

Reiy depends on the Sui venue landscape in one structural way: it reads a canonical on chain price. DeepBook, the native order book, exposes a per pool mid price that a contract can query directly [4], which Reiy uses as the admission reference of Section 5. How a solver then sources liquidity is outside the protocol.

### 2.3 Intents, Solvers, and Batch Auctions

A user signs an *intent*, a desired outcome such as a minimum to receive, and delegates execution to competing *solvers* [3]; batching intents admits a coincidence of wants, where opposing orders clear without price impact and only the residual is sourced externally.

In prevailing designs the intent and all coordination around it live off chain, and only settlement touches the chain; that off chain locus is what affords stable latency and low cost. This frames Reiy’s central question: can an intent be escrowed and enforced on chain, making custody and user protection trustless, while keeping the latency and cost of an off chain market? Reiy answers yes, by escrowing the intent on chain and confining the off chain coordinator to selection and signing. Its second requirement, permissionless

solver participation against the oligopoly tendency of intent markets [2], is made safe by on chain verification (Section 5).

## 3 SYSTEM MODEL

### 3.1 Actors and Roles

The protocol admits two sources of truth. The on chain *Contract* is the authority over custody and verification; the off chain *Fisherman* is the authority over coordination and certification. All other participants are untrusted, and their claims are verified against one of these two authorities. Table 1 states each role and its trust assumption.

**Table 1: Participants, roles, and trust assumptions.**

| Participant | Role                                        | Trust     |
|-------------|---------------------------------------------|-----------|
| Trader      | Originator of an escrowed intent            | Untrusted |
| Solver      | Bonded execution agent                      | Untrusted |
| Fisherman   | Off chain coordinator and certifier         | Bounded   |
| Contract    | On chain custody and verification authority | Trusted   |

The Contract is the source of truth for state and safety: it admits intents, verifies the certificate, and enforces the payout floors and the atomicity of settlement. The Fisherman is the source of truth for ordering and selection; it is the sole off chain coordinator and subsumes the auctioneer, keeper, and relayer roles of earlier designs. The DeepBook mid price is consumed as a verification input, not as a trusted party.

### 3.2 Trust and Threat Model

Reiy is trust minimized rather than trustless. The Contract is trusted as the on chain rule enforcer; the Fisherman is trusted only for liveness, its authority bounded by what the Contract accepts; traders, solvers, and external liquidity are untrusted and verified on chain. A malicious or offline Fisherman can therefore stall progress but never cause loss of funds, since custody and every user protection live in the Contract; Section 8 states the resulting guarantees formally.

The remaining adversaries reduce to this separation. Solver collusion is countered by permissionless entry, since any outside party can undercut a colluding group; manipulation of the reference price is bounded by restricting trading to supported pairs and validating the price at read time; a winning solver that fails to settle is penalized by bond slashing and superseded through fallback.

### 3.3 Design Goals and Properties

Reiy is built to guarantee the following properties, each established by the mechanism in later sections.

- **Non custodial.** Escrow lives in the Contract and off chain authority is bounded, so user funds are never under a coordinator's control.
- **User protected execution.** An SBBO admission floor and a per intent protected minimum are enforced on chain, and they hold even after fees are deducted.
- **Atomic settlement.** A batch settles in one PTB through the hot potato pattern, so it either completes fully or reverts.
- **Permissionless solver competition.** Any bonded party may solve, with no whitelist, which directly answers the oligopoly risk of intent markets [2].
- **Liveness and no stuck funds.** Epoch advance and fallback are permissionless, so no party can freeze the protocol.
- **Verifiable and auditable.** Events expose committed and actual values so indexers and auditors can reconstruct the quality of every batch.

## 4 SYSTEM OVERVIEW

### 4.1 Architecture

Reiy is split between an on chain core and an off chain coordinator, the two sources of truth of Section 3, surrounded by supporting services that hold no authority. The Contract escrows intents, verifies certificates, and enforces the floors and fees. The Fisherman groups intents, runs the auction, and signs certificates. Around them, an Indexer decodes on chain events into queryable state and an OrderBook API serves quotes and open intents to solvers. None of these supporting services can move funds or override a protection; the system's guarantees never depend on them.

### 4.2 Intent Lifecycle

A trade moves through one epoch as the sequence of Figure 2. The Contract keeps the design small, escrowing intents, verifying certificates, and settling without any on chain phase machine; the phase sequencing itself is orchestrated off chain by the Fisherman. Intents are collected behind their SBBO floors, solvers build settlement plans over the batch, the Fisherman ranks the valid ones and certifies a winner, and that winner settles the certified plan in a single PTB. Section 5 defines each step. If no valid solution arrives or the winner fails to settle in time, a permissionless fallback advances the epoch and returns the affected intents to the next batch, so the protocol never stalls.

### 4.3 Where Each Guarantee Lives

Each design goal of Section 3.3 maps to a point in this lifecycle: the SBBO floor protects users at admission (Section 5.2), permissionless competition and ranking occur while solving and selecting (Section 5.3), and certificate verified atomic settlement enforces the protected minimums (Sections 5.4 and 5.5). The economic model of Section 7 aligns incentives without touching protected funds.

## 5 PROTOCOL DESIGN

The protocol is built from five core constructs: the *intent*, the *SBBO floor*, the *solver*, the *certificate*, and the *settlement transaction*, governed by a *uniform clearing price* that keeps each batch fair. We define each in turn and then describe epoch progression. The construct names and parameters follow the Move implementation.

### 5.1 Intents

**Definition 1** (Intent). *An intent is a shared object `Intent<Sell, Buy>` that escrows a trader's sell asset together with the terms under which it may be filled.*

It records the locked `sell_balance`, the effective `min_amount_out`, the SBBO floor and reference mid price fixed at admission, the slippage tolerance, a partial fill flag, a target epoch, and a deadline; it also retains the original sell amount and minimum so that proportional minimums remain comparable across partial fills. Escrow is non custodial: until settlement or cancellation the asset remains in the object, and only its owner may cancel, recovering the full balance.

An intent is created through the auction module, which reads the DeepBook mid price [4], computes the SBBO floor, verifies that the pair is supported and that the slippage tolerance is within the configured maximum, and admits the intent only if its minimum clears the floor.

### 5.2 The SBBO Floor

**Definition 2** (SBBO floor). *The SBBO floor, for Sui Best Bid and Offer, is the on chain lower bound that an intent's minimum must clear at admission, derived from the live market price so that no intent can be admitted with an unreasonably low minimum.*

Let  $x$  be the sell amount,  $P$  the DeepBook mid price [4] scaled by  $10^9$ , and  $\sigma$  the slippage tolerance in basis points. The expected output is taken in the pool's direction,

$$out = \begin{cases} \lfloor x \cdot P / 10^9 \rfloor & \text{selling base for quote,} \\ \lfloor x \cdot 10^9 / P \rfloor & \text{selling quote for base,} \end{cases}$$

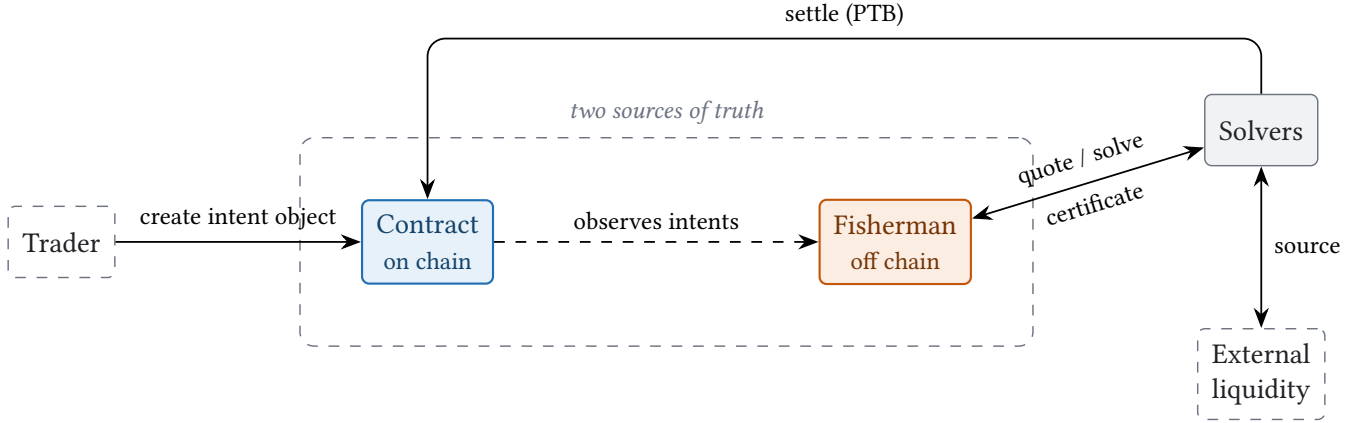


Figure 1: The main flow between the two sources of truth.

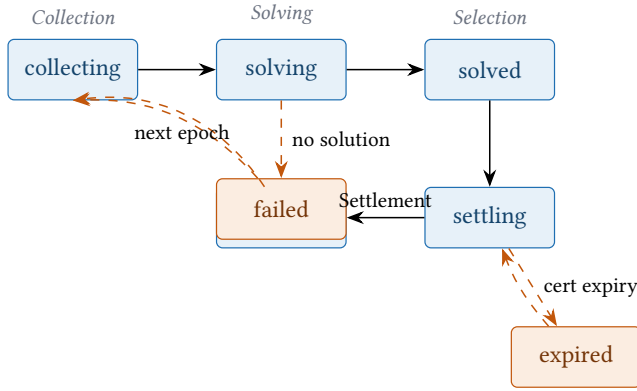


Figure 2: Per epoch intent lifecycle (Fisherman round states; settled and the dashed paths return intents to the next epoch).

and the admission floor applies the tolerance,

$$F = \left\lceil out \cdot \frac{10,000 - \sigma}{10,000} \right\rceil.$$

An intent is admitted only if  $\text{min\_amount\_out} \geq F$ . The tolerance is bounded by a configurable maximum, by default 500 basis points with a hard ceiling of 2,000 basis points, and the mid price must exceed a configured minimum or the read aborts. Because  $F$  is computed from a live on chain reference and stored on the object, neither the solver nor the coordinator can later admit a fill that ignores the market.

### 5.3 Solvers

**Definition 3 (Solver).** A solver is an untrusted, bonded agent that proposes how a batch of intents should be filled, competing permissionlessly to have its proposal selected.

Participation is open: any party posting the minimum bond of Section 7 may compete, and the protocol places no constraint on how a solver obtains liquidity. A solver may clear opposing intents against one another as a coincidence of wants, which carries no price impact, and source any residual externally; these strategies are economic choices internal to the solver and outside the protocol.

A solver expresses its proposal as a *solution*: per intent, a fill amount, a gross payout, and the protected minimum it commits to, together with a single *score* that measures the solution’s total quality in one comparable unit. A solution is valid only if its epoch matches the batch, its per intent vectors are well formed and unexpired, and each entry satisfies

$$\text{gross payout} \geq \text{protected min} \geq \text{user minimum},$$

with a positive fill not exceeding the intent’s remaining amount. Within a directed market the payouts must additionally form a uniform clearing price (Section 5.4), so solvers compete by raising that price, not by discriminating between users. Valid solutions are ranked by score and the highest wins, ties broken deterministically. Because admission is permissionless and ranking depends only on score, any outside solver can win by offering strictly better execution, which is the structural countermeasure to the entry barriers that drive intent markets toward oligopoly [2].

### 5.4 Fairness: the Uniform Clearing Price

**Definition 4 (Clearing price).** A clearing price is a single exchange rate at which every intent on the same directed market in a batch settles, so that no user is filled at a worse price than another on that market.

The per intent floors of Section 5.2 protect each user in isolation but say nothing about how users on the same market fare relative to one another, which would let a solver cherry

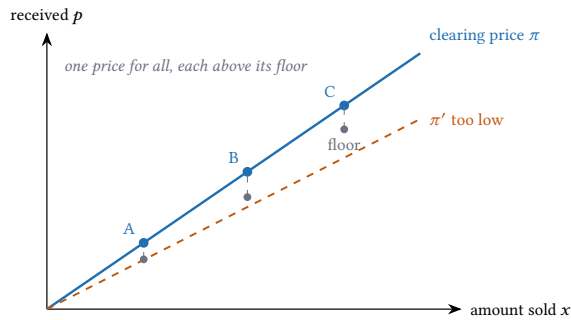
pick: fill one order at an excellent price and another at only its floor. Reiy removes that discretion with a uniform clearing price, the fairness rule of CoW Protocol [3]. Let intent  $i$  sell  $x_i$  and receive  $p_i$ . A solution settles a directed market at a single price  $\pi$  when

$$p_i = x_i \pi \quad \text{for every intent } i \text{ on the market,}$$

equivalently when  $p_i/x_i$  is equal across them. Two conditions bind  $\pi$ . It must clear every floor,  $p_i \geq \text{protected min}_i$ , so  $\pi \geq \max_i(\text{protected min}_i/x_i)$ : the tightest floor per unit sold sets the lowest admissible price. And competition drives it upward, since a solver wins by delivering more total surplus,

$$\sum_i (p_i - \text{protected min}_i) = \pi \sum_i x_i - \sum_i \text{protected min}_i,$$

which grows with  $\pi$ . The winning solution is the highest uniform price the batch's sourced liquidity can support.



**Figure 3: Uniform clearing price.** Every intent on a market lies on one price ray  $\pi$ , and the gap from each user's floor up to the ray is its surplus. A lower price  $\pi'$  is infeasible because it would breach the tightest floor.

The Contract enforces uniformity on chain. For the intents of one directed market it checks that  $p_i/x_i$  is equal within a small tolerance, a cross multiplication  $p_i x_j = p_j x_i$  that avoids division, alongside the per intent floor check. Fair pricing is therefore not a matter of solver honesty but a verified on chain invariant: every user on a market receives the same price, and that price still clears each user's SBBO floor.

This is the uniform clearing price of a batch auction, produced here by permissionless competition rather than assumed after the fact. A fair combinatorial auction shows that competition among solvers can construct such a fairness benchmark endogenously, from the solvers' own bids, without the protocol needing to know the fair price in advance [1].

## 5.5 Certificates

**Definition 5 (Certificate).** A certificate is the off chain coordinator's signed authorization of a specific settlement, and the sole credential that lets a solver act on escrowed funds.

The Fisherman signs a SolutionMessage with its Ed25519 key, binding the protocol state and config object identifiers, the key version, the epoch, the solver address, the traded types, and the per intent fills, gross payouts, and protected minimums, together with an expiry. The certificate carries no authority of its own; it is meaningful only because the Contract reconstructs the same message from on chain state and verifies the signature, so the coordinator can authorize a plan but cannot alter the rules that bind it.

## 5.6 Settlement via Programmable Transaction Blocks

**Definition 6 (Settlement).** A settlement is the single Programmable Transaction Block (PTB) in which the winning solver redeems a certificate, releasing escrow and delivering the certified payouts atomically.

Settlement is where the certified plan meets on chain enforcement, and where Reiy exploits the structure of Sui transactions. The winning solver executes the whole batch in one PTB (Figure 4), so verification, the release of escrow, and the delivery of payouts all commit or all revert.

Two properties of Sui make this possible. A PTB threads the typed result of one Move call into the next, so a value produced by verification flows through the calls that release escrow and deliver payouts without leaving the transaction. The *hot potato* pattern makes that threading mandatory: a Move value whose type has none of copy, drop, store, or key cannot be copied, discarded, or stored, only unpacked by a function in its defining module, so a value created mid PTB obligates a later call to consume it or the transaction is ill formed. Settlement thus becomes a type system guarantee rather than a matter of trust.

Concretely, `take_authorized_intent` returns the escrowed sell coin and a `SettlementReceipt` with no abilities, a true hot potato consumable only by `settle_intent`. The moment a solver takes an intent's escrow it holds a receipt it cannot drop, store, or move outside the transaction, so it must call `settle_intent` with the exact certified payout in the same PTB. No escrow leaves the contract without a matching settlement, for every intent in the batch at once.

Three checks make this safe. First, `verify_solution` rebuilds the message from on chain state, so the epoch, object ids, and token types cannot be forged, and it checks the sender is the named solver, the certificate has not expired,

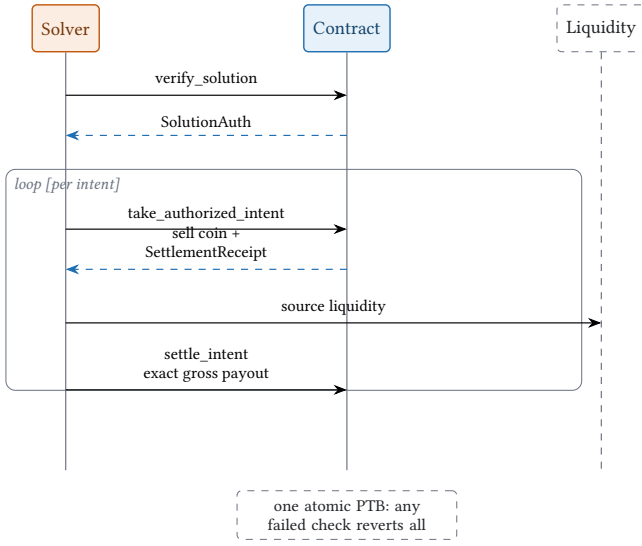


Figure 4: The single PTB settlement sequence.

and the Ed25519 signature matches the configured coordinator key. It returns a `SolutionAuth` that fixes the exact list of intents and payouts.

Second, each `take_authorized_intent` call advances through the certified list in order, and requires that the intent id matches the next entry, that the intent’s `target_epoch` equals the certificate epoch, that it has not expired or already been partially filled this epoch, and that the committed `protected_min` is at least the intent’s own minimum. It returns the escrowed sell coin and the `SettlementReceipt` hot potato described above, which obligates the matching `settle_intent` call.

Third, `settle_intent` requires the delivered buy coin to equal the certified gross payout exactly, confirms the solver is active and bonded, deducts fees so the net stays at or above the protected minimum, and transfers the net to the trader. Any failed check reverts the whole PTB, so the trader’s floor holds even after fees.

## 5.7 Epochs and Partial Fills

The on chain state keeps only an epoch counter, a start timestamp, the set of intents partially filled this epoch, and fee aggregates. There is no on chain phase machine; epoch progression is permissionless. Anyone may call `advance_epoch` once the minimum collection time has passed, which increments the epoch and clears the partial fill set, so the protocol cannot be frozen by a stalled coordinator.

A partial fill consumes part of an intent and rolls the remainder forward. The proportional minimum for a fill of size  $f$  on an intent with original sell amount  $x_0$  and original

minimum  $m_0$  is

$$m_{eff} = \lceil m_0 \cdot f / x_0 \rceil,$$

computed against the original amounts so that successive partials stay comparable. The residual advances its `target_epoch` by one and re enters the next batch, and the contract forbids two fills of the same intent within one epoch.

## 6 THE FISHERMAN COORDINATOR

The Fisherman is the off chain source of truth for coordination. It holds no user funds and cannot override any on chain check; it exists to make solver competition fast and to produce the certificates the contract verifies.

### 6.1 Services

The coordinator is organized as a small set of services. An Indexer ingests the Reiy contract events from Sui, and an Orderbook exposes the open intents drawn from that index. A Quote service fans pre trade quote requests to enabled solvers and ranks the replies. An Auction service groups open intents into rounds by token pair and epoch. A Driver calls solver endpoints with bounded timeouts, a Scoring service validates and ranks the responses, and a Certificate service signs the canonical settlement message. A Settlement watcher marks expired certificates so a round can be retried.

### 6.2 Pre-Trade Quotes

Before committing an intent, a trader may request an indicative quote. The Fisherman normalizes the request and fans it to every enabled solver in parallel, collects the replies within a bounded window, discards the late or invalid ones, and returns the best valid quote, as shown in Figure 5. The flow is purely informational: it touches no escrow and binds no party, and the trader proceeds to create an intent only if the quote is acceptable.

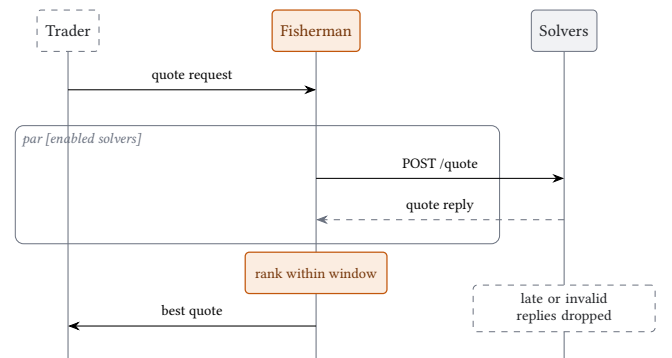


Figure 5: Pre-trade quote flow: the Fisherman fans a request to enabled solvers in parallel and returns the best valid reply.



### 6.3 Round and Certificate Lifecycle

A round moves through the states collecting, solving, solved, settling, and settled, with expired and failed for the unhappy paths. Open intents are grouped into a collecting round, defaulting to four intents per round and never more than ten. Once the collection window passes, the round moves to solving, enabled solvers are asked to solve, and the highest scoring valid plan is selected. The Certificate service then signs the solution, and when the winning plan carries a Coincidence of Wants counter leg it signs both legs as a pair. The certificate itself runs issued, settling, settled, and is marked expired or reissued if the winner does not settle in time. An administrator endpoint reissues a certificate to the next candidate, and because on chain settlement is always available for a live epoch, a fresh certificate can settle immediately.

### 6.4 API Surface

The external interface follows the Coincidence of Wants convention with `/quote`, `/solve`, `/reveal`, `/settle`, and `/notify`, while the wire model is Sui native, carrying object ids, Move type names, fills, protected minimums, and certificate expiry. Read endpoints expose the orderbook, active auctions, and signed certificates, and an event stream lets integrators follow rounds without speaking to the indexer directly. Quote collection is bounded by a short window, by default a little over a second, and replies that miss it are reported as failed rather than allowed to stall the round.

## 7 ECONOMIC MODEL

The economic model has three layers: the fee the protocol charges, the revenue model that fee produces, and the REIY token that secures and rewards the network. We present the mainnet design; the current testnet contracts use a simpler in kind fee split and a SUI bond, which mainnet supersedes.

### 7.1 Fee Structure

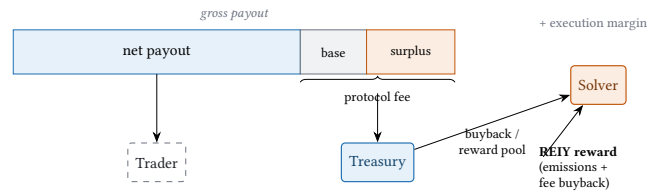
Reiy charges a fee only on a settled intent, taken from the gross payout and structured so the trader's net never falls below the protected minimum. The fee has two components. A *base fee* is a flat rate on the notional, set at 0.02% by default and 0.01% for correlated or stable pairs; it ensures the protocol earns a small, predictable amount on every trade even when competition drives surplus to zero. A *surplus fee* then takes a share, 20% by default, of the surplus delivered above the user's protected minimum, capped at 0.15% of notional. The total is capped at 0.25% of notional, so the net payout always remains at or above the protected minimum.

Figure 6 shows how a settled payout is partitioned. The base and surplus fees accrue to a per asset protocol treasury, denominated in the traded asset. The solver is rewarded

**Table 2: Fee parameters (governable initial values).**

| Parameter             | Default | Basis       |
|-----------------------|---------|-------------|
| Base fee (standard)   | 0.02%   | of notional |
| Base fee (correlated) | 0.01%   | of notional |
| Surplus fee           | 20%     | of surplus  |
| Surplus fee cap       | 0.15%   | of notional |
| Total fee cap         | 0.25%   | of notional |

separately, in REIY rather than from the fee, and additionally retains any execution margin it captures above the certified payout. Tying the surplus fee to delivered surplus makes the protocol's revenue grow precisely when users are filled better than their floor, never at their expense.



**Figure 6: Settlement payout split: the trader keeps the net, the protocol fee funds the treasury, and the solver is rewarded in REIY plus any execution margin.**

### 7.2 Business Model

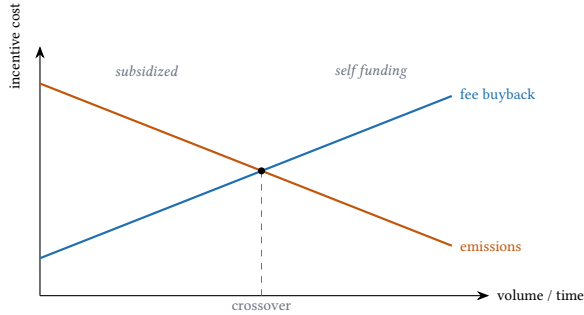
Reiy monetizes a slice of the surplus a competitive solver network delivers, not the trader's principal. Because the fee is bounded so the net never drops below the protected minimum (Section 7.1), revenue comes only from value the network created. The business is positive sum: Reiy earns only when a trader is filled better.

Revenue has two hedging streams. The *base fee* scales with settled volume and holds a floor even when competition drives surplus toward zero; the *surplus fee* scales with delivered price improvement and peaks when execution is best. As competition sharpens, per trade surplus thins but volume grows, so revenue does not collapse in the equilibrium that best serves users.

Unit economics are favorable: revenue per trade is capped at 0.25% of notional while the protocol's marginal settlement cost is near zero, since the solver pays gas and the coordinator is a fixed cost. The real cost base is not settlement but the incentive that keeps solvers competitive.

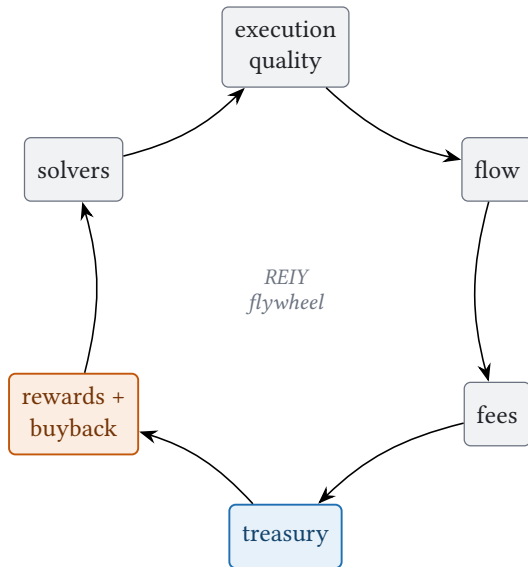
That incentive is paid in REIY, not from the fee, so early growth runs ahead of revenue: emissions bootstrap a competitive solver set before fee revenue is material, and solvers earn REIY on top of the execution margin they capture. As

volume grows, treasury funded buyback replenishes the reward pool and emissions taper. The crossover, where buyback covers the rewards needed to retain solvers, marks the shift from a subsidized to a self funding network (Figure 7).



**Figure 7: Bootstrap to self funding. Scheduled emissions fund solver rewards early; fee funded buyback overtakes them at the crossover, after which real revenue sustains incentives.**

Value compounds (Figure 8): better execution attracts flow, flow yields fees, fees fund rewards and buyback, and deeper rewards draw more solvers. REIY captures the loop, with fee funded buyback underpinning its value, solver bonds locking its supply, and holders governing the parameters. A treasury of real traded assets backs the token, not the reverse.



**Figure 8: The value flywheel: each stage funds the next, and deeper solver competition feeds back into execution quality.**

The sensitivities are bounded. Surplus varies with market conditions, which is why the base fee stabilizes revenue; fees

accrue in a basket of assets, requiring a treasury conversion policy; and the crossover depends on volume growth, so until it is reached the reward pool leans on emissions and their dilution. Each is a governable parameter, set conservatively under a multisig at launch and handed to REIY governance as the network matures.

### 7.3 The REIY Token

REIY is the work and governance token of the network, with three utilities. First, it is the solver *bond*: a solver must lock REIY to compete, and the stake is slashable, so participation carries skin in the game and a failed settlement has a cost. Second, it is the solver *reward*: solvers earn REIY in proportion to the surplus and score they deliver, which directs emissions toward execution quality rather than raw volume. Third, it is the *governance* right: over time REIY holders govern the protocol parameters, the supported pairs, the treasury, and the emission schedule.

Reward funding and value accrual follow the loop of Section 7.2. The treasury is initially held by an AdminCap under a multisig, transitioning to on chain governance as the network matures.

### 7.4 Bonding and Slashing

The bond turns the right to compete into an accountable one. To register, a solver locks at least the minimum bond in REIY; the stake gates participation and is the value at risk if the solver misbehaves.

The slashable event is a *failure to settle*. Recall from Section 5 that an invalid settlement cannot cause loss, because the PTB simply reverts and no escrow moves; the only way a solver can harm the batch is to win a certificate and then not settle it before the certificate expires, which stalls the intents it was assigned. This failure is provable from on chain state alone: the certificate binds the solver, the epoch, and the expiry, so an assigned round left unsettled past expiry is unambiguous evidence of default.

Slashing is not automatic, since a contract acts only when invoked; it is triggered by a permissionless *reporter*. Any party may submit the expired, unsettled certificate to the Contract, which verifies the default from on chain state before acting, so a false report is impossible and enforcement never depends on the coordinator. The reporter is paid a bounty out of the slashed bond, sized to cover gas and leave a margin, which makes catching defaults profitable for any watcher rather than a duty that falls to no one.

The remaining bond, after the bounty, flows to the treasury and reward pool, redistributing the cost of grieving to the honest solvers who keep settling. A slash takes a governable fraction of the bond and escalates with repeated defaults until a persistently failing solver is deregistered and must



re-bond. Since a default only rolls the affected intents to the next epoch through the fallback of Section 5.6, the bond need only make repeated griefing unprofitable, not collateralize the full trade value.

## 8 SECURITY AND PROPERTIES

The design goals of Section 3.3 follow from the mechanism just described. We summarize how each is enforced and why safety does not depend on the coordinator.

**Property 1** (Non-custodial). *User funds are never under the control of any off chain party; escrow resides solely in the shared Intent object.*

The only on chain path that releases the asset is a certified settlement that passes every check, or a cancellation by the owner.

**Property 2** (User-protected execution). *A settled trader never receives less than the protected minimum, even after fees.*

Two floors enforce this: the SBBO floor at admission and the per intent protected minimum at settlement, where the fee logic additionally requires the net after fees to stay at or above that minimum.

**Property 3** (Fair pricing). *All intents on the same market settle at one clearing price that clears every floor, so no user is cherry picked.*

The Contract verifies that the payout to sell ratio is equal across a market by cross multiplication, so uniform pricing is an on chain invariant rather than a matter of solver discretion (Section 5.4).

**Property 4** (Atomic settlement). *A batch either completes in full or reverts in full.*

The non droppable SettlementReceipt forces every intent taken in a PTB to be settled in the same PTB at its exact certified payout.

**Property 5** (Permissionless competition). *Any bonded solver can win by score alone, with no whitelist.*

This is the direct countermeasure to the entry restriction and oligopoly that a formal analysis identifies as the central weakness of intent markets [2].

**Property 6** (Liveness). *Neither a silent solver nor an offline coordinator can freeze the protocol.*

Epoch advancement and certificate reissuance are permissionless or routine operations, so the worst a faulty Fisherman can do is delay progress.

This yields a clean separation: the Fisherman governs speed and selection, but the contract alone governs custody and correctness. A malicious coordinator cannot move funds, cannot admit a sub floor intent, cannot pay a trader below their minimum, and cannot make a solver win without a signature the contract verifies. Loss of funds is impossible without a flaw in the contract itself, which narrows the trusted surface to the audited Move modules.

## 9 CONCLUSION

Reiy brings intent based, solver competitive execution to Sui without asking users to trust an off chain operator with their funds. Traders escrow an intent behind an on chain SBBO floor, permissionless solvers compete to fill it, and a certificate signed off chain is verified on chain before any asset moves, so settlement is atomic and the user's minimum always holds. The protocol does not replace the venues Sui already has; it turns their fragmented liquidity into depth a single trade can reach. By keeping custody and correctness on chain while letting coordination and competition run off chain, Reiy aims to make best execution a default property of trading on Sui rather than a manual effort.

## REFERENCES

- [1] Andrea Canidio and Felix Henneke. 2025. Fair Combinatorial Auction for Blockchain Trade Intents: Being Fair without Knowing What Is Fair. <https://doi.org/10.48550/arXiv.2408.12225> arXiv:2408.12225 [econ.TH]
- [2] Tarun Chitra, Kshitij Kulkarni, Mallesh Pai, and Theo Diamandis. 2024. An Analysis of Intent-Based Markets. <https://doi.org/10.48550/arXiv.2403.02525> arXiv:2403.02525 [cs.GT]
- [3] CoW DAO. [n. d.]. Documentation - CoW DAO. <https://docs.cow.fi/>.
- [4] Mysten Labs. [n. d.]. Query the Pool | Sui Documentation. <https://docs.sui.io/onchain-finance/deepbookv3/contract-information/query-the-pool>.
- [5] Mysten Labs. [n. d.]. Sui DeFi Stack | High Performance DeFi Infrastructure. <https://www.sui.io/defi>.